Latest updates: https://dl.acm.org/doi/10.1145/3636243.3636247

PDF Download
3636243.3636247.pdf
15 March 2026
Total Citations: 25
Total Downloads: 524

Published: 29 January 2024

Citation in BibTeX format

ACE 2024: Australian Computing Education Conference
January 29 - February 2, 2024
NSW, Sydney, Australia

RESEARCH-ARTICLE

More Robots are Coming: Large Multimodal Models (ChatGPT) can Solve Visually Diverse Images of Parsons Problems

IRENE HOU, Temple University, Philadelphia, PA, United States

OWEN MAN, Temple University, Philadelphia, PA, United States

SOPHIA METTILLE, Temple University, Philadelphia, PA, United States

SEBASTIAN GUTIERREZ, Temple University, Philadelphia, PA, United States

KENNETH ANGELIKAS, Temple University, Philadelphia, PA, United States

STEPHEN MACNEIL, Temple University, Philadelphia, PA, United States

Open Access Support provided by:

Temple University

ACE '24: Proceedings of the 26th Australasian Computing Education Conference (January 2024)
https://doi.org/10.1145/3636243.3636247
ISBN: 9798400716195

More Robots are Coming: Large Multimodal Models (ChatGPT) can Solve Visually Diverse Images of Parsons Problems

Irene Hou
Temple University
Philadelphia, PA, USA
irene.hou@temple.edu

Owen Man
Temple University
Philadelphia, PA, USA
owen.man@temple.edu

Sophia Mettille
Temple University
Philadelphia, PA, USA
sophia.mettille@temple.edu

Sebastian Gutierrez
Temple University
Philadelphia, PA, USA
guts@temple.edu

Kenneth Angelikas
Temple University
Philadelphia, PA, USA
kenneth.angelikas@temple.edu

Stephen MacNeil
Temple University
Philadelphia, PA, USA
stephen.macneil@temple.edu

ABSTRACT

Large language models are reshaping computing education. Based on recent research, these models explain code better than students, answer multiple choice questions at or above the class average, and generate code that can pass automated tests in introductory courses. In response to these capabilities, instructors have quickly adjusted their courses and assessment methods to align with shifting learning goals and the increased risk of academic integrity issues. While some scholars have advocated for the integration of visual problems as a safeguard against the capabilities of language models, new multimodal models now have vision and language capabilities that may allow them to analyze and solve visual problems. In this paper, we compare the large multimodal model (LMMs) GPT-4V with Bard, an LLM that uses Google Lens for text recognition. We find that LMMs, which have learned both pixel features (from images) and text features (from prompts) in the same embedding space, performed substantially better than Bard which uses a piecemeal approach. With a specific focus on Parsons problems presented across diverse visual representations, our results show that GPT-4V solved 96.7% these visual problems, struggling minimally with a single Parsons problem. Conversely, Bard performed poorly by only solving 69.2% of problems, struggling with common issues like hallucinations and refusals. These findings suggest that merely transitioning to visual programming problems might not be a panacea to issues of academic integrity in the generative AI era.

CCS CONCEPTS

• **Social and professional topics → Computing education**.

KEYWORDS

Generative AI, LLMs, GPT-4V, ChatGPT, Bard, Parsons Problems, visual programming problems, computing education

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
ACE 2024, January 29–February 02, 2024, Sydney, NSW, Australia
© 2024 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 979-8-4007-1619-5/24/01…$15.00
https://doi.org/10.1145/3636243.3636247

ACM Reference Format:
Irene Hou, Owen Man, Sophia Mettille, Sebastian Gutierrez, Kenneth Angelikas, and Stephen MacNeil. 2024. More Robots are Coming: Large Multimodal Models (ChatGPT) can Solve Visually Diverse Images of Parsons Problems. In *Australian Computing Education Conference (ACE 2024), January 29–February 02, 2024, Sydney, NSW, Australia.* ACM, New York, NY, USA, 10 pages. https://doi.org/10.1145/3636243.3636247

1 INTRODUCTION

Large language models (LLMs) are incredibly powerful tools with many exciting use cases emerging in computing education contexts. These models are now capable of generating code [2, 7, 11, 53, 70], answering multiple-choice questions [58, 60], and even solving Parsons problems [54]. While students can use these models to obtain high-quality explanations of code [42, 48, 49], the ability for students to use these tools to complete assignments or pass exams has raised numerous concerns from computing education researchers and practitioners about the effects these tools will have on assessment [41, 46, 51, 52, 74] and help seeking [30].

To address these concerns related to academic integrity, researchers have advocated for adopting proctored exams [36, 52, 55, 67, 74] and visual programming problems [12] as potential barriers for students seeking these tools to cheat. However, drawbacks of in-person proctored exams and code interviews include scalability and known biases around test-taking [10, 29, 71]. AI proctoring systems for online exams resolve scalability challenges but bring about new issues related to accessibility, violations of privacy and autonomy, and the potential for bias [6, 9, 26]. On the other hand, visual representations, such as photos of assignments and 'prompt problems' [12], are being explored as another way to mitigate the unauthorized use of generative AI by making it challenging to directly feed problems into LLMs. Parsons problems are similarly challenging to input because existing user interfaces do not lend themselves to simple copy and pasting. While these approaches rely on visual representations to circumvent existing LLMs, LMMs, such as GPT-4V, are now capable of processing text and image data.

In this paper, we investigate the vision capabilities of two popular generative AI models (i.e.: GPT-4V [1] and Bard [2]). GPT-4V is a large multimodal model (LMM) which was trained on both text and pixel features (from images). Bard is a large language model (LLM) that

[1] https://cdn.openai.com/papers/GPTV_System_Card.pdf
[2] https://bard.google.com/

uses Google Lens for text recognition. As a first step in understanding their capabilities to solve visual problems, We evaluated their performance on multiple Parsons problems across diverse visual representations. We created a dataset of Visual Parsons problems based on existing literature [54], then represented each Parsons problem across diverse visual formats based on common Parsons Problem generation tools include JS-Parsons [28, 32, 33, 64], Runestone [17, 19, 21, 72], Epplets [38, 39], and EvoParsons [1, 24]. We then prompted GPT-4V and Bard with pre-engineered prompts to evaluate their ability to solve these images of visually diverse Parsons problems.

This paper investigates the following research questions:

RQ 1: How effectively do large multimodal models generate solutions to Parsons problems?

RQ 2: How do visual variations of Parsons problems, recreated using different generation tools, influence the problem-solving performance of large multimodal models?

Our results suggest that new large multimodal models (LMMs) are capable of solving Parsons problems across multiple visual representations with GPT-4V successfully solving 96.7% of the problems. This paper contributes a critical new capability to the growing list of capabilities for LLMs, and now LMMs. This new capability is critical because it further threatens traditional assessment as students can use these models to now solve visual programming problems. However, it is important to note that Bard was only able to solve 69.2% of the problems. Furthermore, during our prompt engineering process, performance varied widely depending on the prompt. This reiterates the effect that the model and prompt have on performance.

2 RELATED WORK

2.1 Parsons Problems

Parsons problems are a type of programming puzzle where learners are given jumbled code snippets and must arrange them in the correct logical sequence rather than producing the code from scratch [14, 16, 50]. This approach has been explored and implemented in various educational settings, contributing to a deeper understanding of programming concepts [13, 16–18, 20]. Parsons problems are often used to help novice learners learn early programming concepts, such as in CS1 and CS2 courses, by focusing on logic and program structure over the nuances of syntax [50].

Based on a recent ITiCSE working group focusing on Parsons problems, the most popular Parsons problem language was Python, and the most commonly targeted concepts in Parsons problems included loops, conditionals, and lists and arrays [16]. Additionally, many variations of Parsons problems have been introduced to students: problems with distractors (code blocks not required for the solution) [13], 'faded' Parsons problems (code blocks that include blanks) [27], as well as "two-dimensional" problems, which require students to practice proper indentation [34]. Several Smart Learning Content providers (SLC) [5] have also created varied tools on their platforms for instructors to create and assess their own Parsons problems. These tools present Parsons problems in various visual formats, ranging from mobile apps [31] to online problem generators [15, 17, 31].

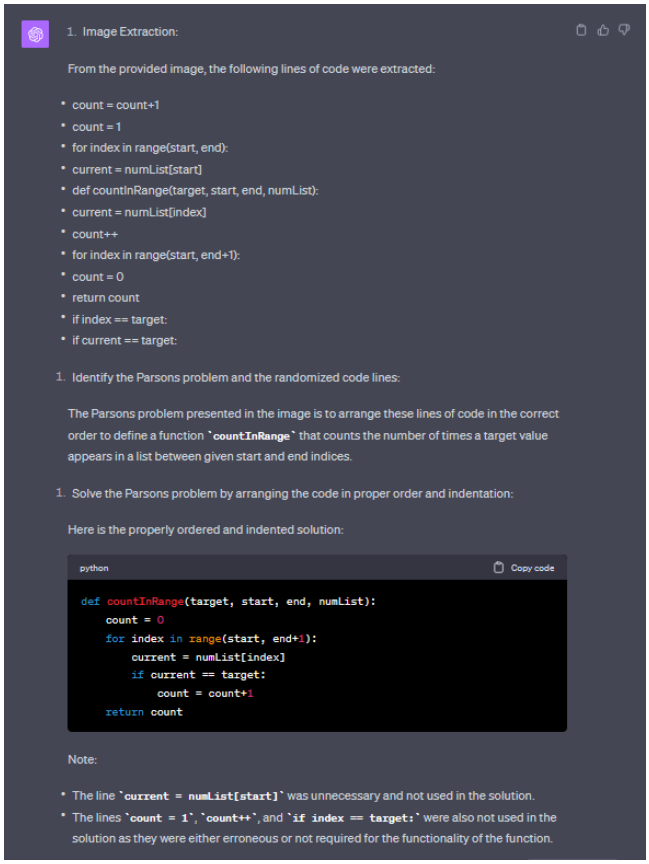

Figure 1: A screenshot of GPT-4V getting the correct answer for an image of a Parsons problem (EvoParsons Haynes-Magyar2022figure4). The response breaks down the problem solving process and even identified the distractors.

A recent study examined the performance of LLMs at solving Parsons problems drawn from a review of the literature [54]. Their analysis, which focused on OpenAI Codex, showed that the model was only able to solve 51% of the problems. Consequently, it is possible that Parsons problems may pose a challenge for LLMs. Moreover, Parsons problems are often difficult to copy and paste from the user interface into LLMs. Given these two challenges, it is possible that Parsons problems may discourage academic dishonesty given the low performance and need for transcribing the problems. Our research extends this prior work focusing on the effectiveness of the new large multimodal models (LMMs) which can analyze text and image data.

2.2 The Effects of Generative AI on Assessment

Over the last two years, there has been an immense amount of research focusing on the capabilities of generative AI models. Initial research identified exciting new capabilities such as creating programming assignments for instructors [23, 57] and explaining code to students [42, 48, 49]. As additional use cases have emerged

showing how adept generative models are at solving programming problems [53, 70], identifying bugs in code [45], automatically repairing bugs in code [22, 35, 37], and passing multiple choice quizzes [58, 59, 61], researchers and practitioners have become concerned about the implications regarding assessment and academic integrity [3, 41, 74]. Initially, practitioners wanted to ban these models from their classrooms, but have since shifted toward the belief that 'resistance is futile' — that students can and will use these models even if they are prohibited in the class [41]. An interview study with students corroborated these observations with multiple students saying they would use LLMs to help them with anything they considered 'busy work' [74]. This is also partially a reaction to the fact that there is currently no reliable way to differentiate between human and AI-generated content [56].

To address these concerns around assessment, both researchers and practitioners have advocated for a return to heavily proctored exams [36, 67, 74]. While this may partially reduce the impacts of LLMs on assessment, students have shared that they strongly dislike this solution [74] and proctored exams can disproportionately benefit students who are better test takers. Researchers have also devised 'prompt problems' that make it harder for students to use generative AI tools directly [12]. However, with the introduction of large multimodal models, it is unclear whether this solution will continue to address the problem. Research is needed to understand the capabilities of LMMs given their potential impact on assessment.

3 METHODOLOGY

This study aims to investigate the recently deployed large multimodal models (LMMs) that generate natural language output given a text prompt and image. These models, such as Bard and GPT-4V, have not yet been studied in the context of computing education, but they could have significant impacts. Over a year ago, researchers raised the clarion call that 'robots are coming' [23], but in this work we focus on the ability for these new models to interpret and solve visual programming problems. We use Parsons problems as a step toward understanding their capabilities and implications more broadly. To conduct this investigation, we needed to develop a dataset of visual Parsons problems and design a prompt that performs well at solving these problems. In this section, we describe how we developed our dataset, engineered an effective prompt, and formed the process for evaluating the model's performance.

3.1 Visual Parsons Problems Dataset

Given that a dataset of visual Parsons problems does not currently exist, we formed our own dataset guided by recent related research. Based on a recent working group by Ericson et al. [16], we chose four commonly referenced Parsons problem generators, which each varied in visual style. We then drew six Parsons problems from a recent study that assessed LLM performance on text-based Parsons problems [54]. The selected Parsons problems are representative of introductory programming concepts and have all appeared in previous literature.

3.1.1 Four Visually Diverse Representations of Parsons Problems. Our dataset comprises four visually-diverse Parsons problem generation platforms, each offering distinct features and visual designs. Figure 2 displays the visual differences between each Parsons problem generation tool, each option presenting varied colors, fonts, texts, formatting, user interface, and more. This selection of platforms was informed by a recent working group paper by Ericson et al. [16], which gave a comprehensive overview of leading Parsons problem platforms. After reviewing the most cited generation tools, we discarded the platforms that were not open-source or not readily available for public use. The finalized selected platforms included: JS-Parsons [28, 32, 33, 64], Runestone [17, 19, 21, 72], Epplets [38, 39], and EvoParsons [1, 24].

At a high level, all four platforms present one common goal to the user: arrange given code snippets in logical order according to instructions. However, the visual controls for achieving this goal vary. For instance, the JS-Parsons platform presents the problem at the top, with an interface below where the user must drag from a snippet bank on the left into a dedicated solution area on the right, with descriptions above each area. Runestone presents the problem similarly at the top with descriptions of each area, but also adds extra buttons for checking and resetting the work. EvoParsons also includes instructions at the top, yet introduces a unique single area of a list where the snippets all have symbolic icons that indicate dragging vertically to arrange or remove. Finally, Epplets presents a more complex visual landscape, including a full window UI with extraneous elements such as scroll bars, elapsed time, and menu options. This range of visual Parsons problem generators allows us to assess LMM performance in solving across various representations.

For each specified SLC platform, researchers transcribed each Parsons problem into the 4 visual representations. Each Parsons problem was transcribed identically, with scrambled ordering of code blocks remaining the same. Online platforms, such as JS-Parsons, allowed the researchers to input the Parsons problem line by line. For cases like Runestone, where problems are static, researchers utilized the web browser's Developer Tools to modify the HTML/CSS of the page, altering each code block individually to match the selected Parsons problem. Other platforms operated offline, so researchers used photo-manipulation tools to edit each problem's text and instructions, ensuring they mirrored the platform's appearance. Images of the Parsons problems were downloaded in the same PNG format to maintain uniformity in quality, compression, and other attributes.

3.1.2 Six Parsons Problems. To select the Parsons problems for our study, we aimed to include a set that represents the types of problems previously examined in computing education research. Remaining consistent with previous research, we relied on the six problems investigated by Reeves et al. [54] in a recent study of LLMs performance on Parsons problems. While previous work investigated these problems using text-based versions, we extend this work to investigate how large multimodal models perform on this task. The six Parsons problems from their study represent a variety of programming topics, including but not limited to loops, conditionals, and functions. To remain aligned with their study, we also kept all the problems in Python. This choice also corresponds with Python being the most commonly used language for representing Parsons problems [16]. The problems varied not only in terms of their topic but also in their length and complexity, ranging from concise code snippets to more intricate programs.

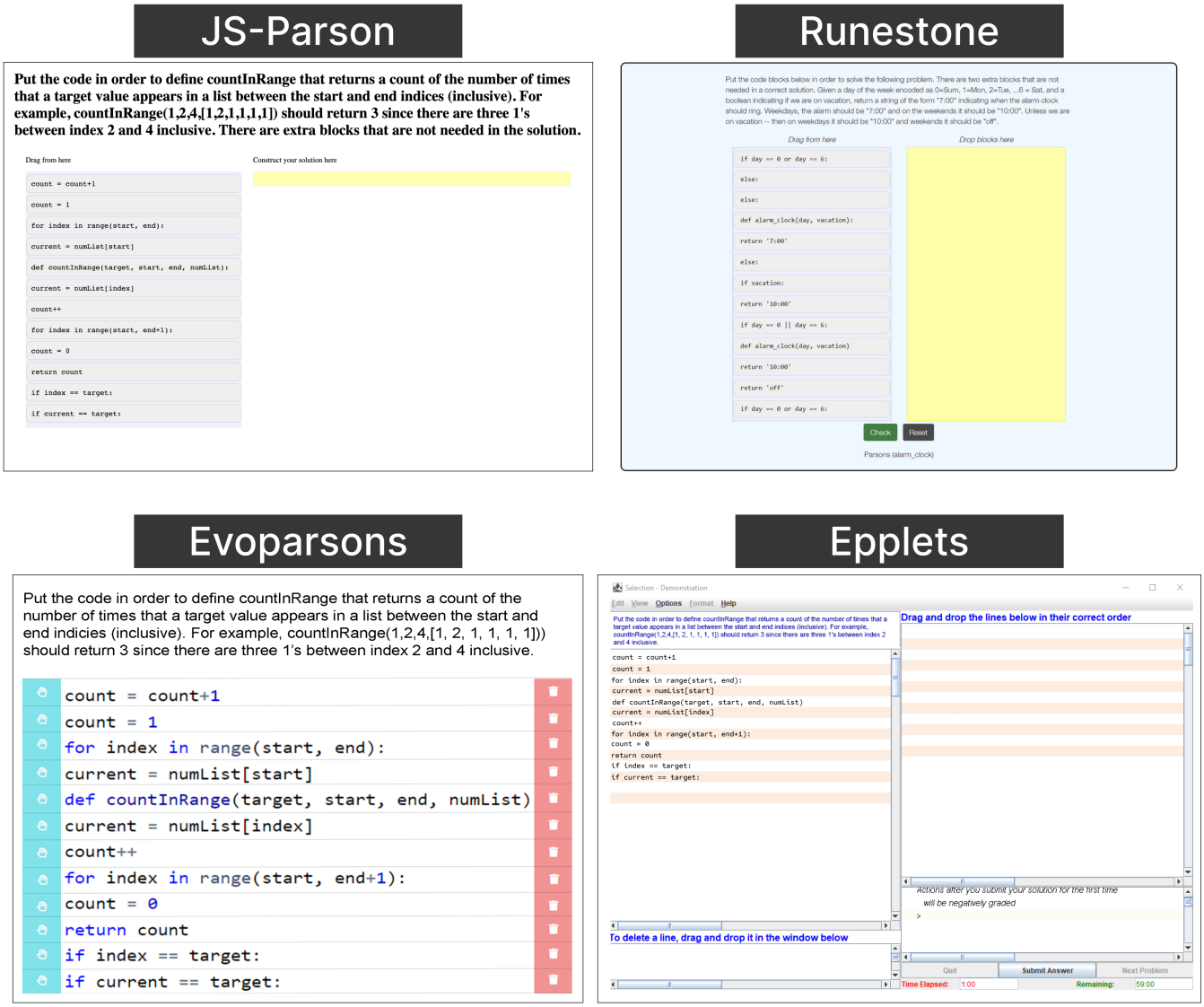

Figure 2: The four visual representations with the same coding example. The representations varied in aspect ratio, background color, fonts, font-size, text placement, and other visual nuances.

3.2 Model Selection

Recently large multimodal models have been developed [25, 43, 44, 69]. While new models are being released on a nearly monthly basis, we selected two highly popular models—OpenAI's GPT-4V and Google's Bard. At the time of writing this paper, GPT-4V is not yet accessible through the API. To ensure a fair comparison, we used the user interface to interact with GPT-4V and Bard.

3.3 Prompt Engineering

Prompt engineering is the iterative process of trying different inputs to an LLM in order to improve the performance. Our process started with the highly generic prompt "solve the Parsons problem in the provided image." For this prompt, performance for GPT-4V was around 50% and even lower for Bard. We issued multiple prompts for 10 random problem and representation pairs. From there, we iterated to improve a few key aspects: 1) reducing high refusal rates, 2) correctly identifying code blocks, and 3) getting the correct ordering and indentation. We were able to address these issues by explicitly explaining in the prompt how Parsons problems work and that not all the code blocks were necessary. We also iterated on a few approaches including chain-of-thought prompting and few-shot prompting where examples were provided. These example-based techniques did not appear to improve performance in our testing. Our process involved developing prompts for both models, but we were only able to achieve high performance with GPT-4V. We selected the prompt that optimized performance for both models.

The resulting prompt consisted of three components which included the 'Context' to inform the model that a screenshot of a coding Parsons problem was provided. The 'Task' which described what a Parsons problem is and an overarching strategy for solving them. Finally, the 'Instructions' explicitly listed the necessary steps in completing the task: 1) reading the text in the image, 2) identifying the task from the text, and 3) putting the snippets in the right order to complete the task. The resulting prompt has been provided below:

Context: *I provided you with a screenshot of a coding Parsons problem.*

Task: *I want you to solve this Parsons problem. Each line of code is blocked and the order is randomized. Sort the lines of code in order and ensure unnecessary lines of code are not part of the answer.*

Instructions:

(1) *Image Extraction: Extract the text from the image*
(2) *Identify the Parsons problem and the randomized code lines*
(3) *Solve the Parsons problem by arranging the code in proper order and indentation*

We used the same prompt for both models. The goal was to ensure a consistent comparison of each model's responses across varied problems and representations.

3.4 Prompting Protocol

All responses were generated over a 3-day period starting on October 25, 2023. In total, there were 240 solution responses generated using GPT-4V and Bard. These 240 responses consisted of the 6 Parsons problems displayed across 4 visual representations with 5 attempts for each of the 2 models.

Generating Responses for GPT-4V and Bard: We prompted each model with an image comprising the problem and representation pair accompanied by the final prompt shown previously. Each prompt and image was used as input five separate times to account for the probabilistic nature of these models. The chat window was closed and a new chat window was opened to avoid responses being affected by the chat history. The first response from the model was saved and analyzed. No follow-up prompting or clarifications were issued. So in cases where the model refused to respond, that was considered a wrong answer. This is important to note, because in our prompt engineering, we observed that follow up clarification prompts can reduce refusal rates and improve performance. When recording the results, three researchers assessed and discussed the validity of each response to ensure agreement.

Strict No-Regeneration Policy: To minimize human variation, we adhered to a strict policy of never regenerating responses. Only the first response from the LMM was considered a solution and no follow-up prompting could be issued.

3.5 Analysis

To evaluate the results, multiple members of the researcher team manually reviewed each output from the LMMs, with at least two members reviewing each output. A correct solution to a Parsons problem involves logically reordering the blocks with the proper indentation. Because each Parsons problem selected had a uniquely correct order, it was not necessary to execute the generated code. We did not observe any disagreements in evaluating the correct solutions between the three raters. In addition to this quantitative analysis, our team also qualitatively reviewed the outputs, looking for interesting outputs and trends. This open-coding method [66] is common when there is a dearth of existing theories to guide the analysis deductively. The goal for this coding was to contextualize the findings with interesting behaviors from the model to shed light on nuances not captured in the quantitative analysis.

4 RESULTS

We evaluated the performance of GPT-4V and Bard with 6 Parsons problems across 4 visual representations and with 5 attempts for each problem-representation pair. This resulted in a total of 120 responses per model with 240 total responses. Based on a McNemar test, which accounted for paired responses, there was a significant performance difference between Bard and GPT-4V conditions (χ^2 = 24.976, df = 1, p < 0.001). The percentage of questions correctly answered for each model was 69.2% for Bard and 96.7% for GPT-4V.

4.1 RQ 1: Correctness of Generated Solutions

4.1.1 Correctness of GPT-4V Generated Solutions. Table 1 summarizes the performance of GPT-4V across 5 attempts for each of the 6 Parsons problems displayed in all 4 visual representations. GPT-4V successfully solved 116 out the of the 120 attempts, achieving an overall success rate of 96.7%. The only mistakes GPT-4V made were on the Haynes-Maygar2022figure2 problem. Shown in Figure 3, GPT-4V hallucinated and rewrote one of the code blocks.

GPT-4V correctly identified the code blocks with the exception of the Haynes-Maygar2022figure2 problem. In that case, it appears highly likely that it identified the code block correctly, given that the hallucination is semantically equivalent.

When distractor code blocks were included, it always explicitly identified them, as shown in Figure 1. Whether or not the explicit identification of distractors played a role in the overall performance, the implication is the model can identify and ignore plausible distractors when included. It is also worth noting that there were syntactic variations in how GPT-4V produced conversational text for each response, although the final code solution remained consistently correct. In some cases, it explained the code blocks, in other cases it explained the reasoning such as what is needed to solve the problem. It seldom generated test cases.

4.1.2 Correctness of Bard Generated Solutions. Bard only produced a correct solution on 69.2% of the attempts. This lower performance appeared to be influenced by a number of factors including refusing to answer (7.5% refusal rate), hallucinating and editing the code blocks (9.6%), producing incorrect sequencing (3.3%), or other problems. Aligned with prior work, we observed cases where Bard struggled with the concept of negation [68].

In terms of hallucinations, Bard had a tendency to edit code blocks. In some cases, Bard converted between single quotes and double quotes or added spacing that was not in the original code block. The correctness of these solutions was assessed on a case by case basis as it often did not affect the correctness of code. In more extreme cases, the code block was rewritten. For example, in the EvoParsons Haynes-Maygar2022figure4 problem, Bard rewrote a code block 4 out of 5 times to be "count +=1" rather than "count = count + 1." Similarly to GPT-4V, there were also variations in the

Problem	**GPT-4V: Number of Correctly Generated Solutions**				**% Total Success**
Image Source	Epplets	JS-Parsons	Runestone	EvoParsons	
Weinmann2021figure1	5/5	5/5	5/5	5/5	100%
Haynes-Maygar2022figure2	5/5	5/5	3/5	3/5	80%
Ericson2022figure2	5/5	5/5	5/5	5/5	100%
Hou2022figure2	5/5	5/5	5/5	5/5	100%
Haynes-Maygar2022figure4	5/5	5/5	5/5	5/5	100%
Ericson2022figure4	5/5	5/5	5/5	5/5	100%
% Total Success	100%	100%	93.3%	93.3%	**96.7%**

Table 1: GPT-4V performance on 6 Parsons problems presented in 4 visual representations with the prompt issued 5 times for each problem and representation pair. Across the 120 responses, GPT-4V was correct 96.7% of the time.

Problem	**Bard: Number of Correctly Generated Solutions**				**% Total Success**
Image Source	Epplets	JS-Parsons	Runestone	EvoParsons	
Weinmann2021figure1	4/5	5/5	2/5	5/5	80%
Haynes-Maygar2022figure2	5/5	5/5	5/5	5/5	100%
Ericson2022figure2	0/5	5/5	4/5	3/5	60%
Hou2022figure2	5/5	5/5	5/5	5/5	100%
Haynes-Maygar2022figure4	0/5	5/5	4/5	1/5	50%
Ericson2022figure4	1/5	0/5	0/5	4/5	25%
% Total Success	50%	83.3%	66.7%	76.7%	**69.2%**

Table 2: Bard performance on 6 Parsons problems presented in 4 visual representations with the prompt issued 5 times for each problem and representation pair. Across the 120 responses, Bard was correct 69.2% of the time.

structure and presentation of the solution. In some cases, Bard explained the produced code and in other cases it generated test cases. It occasionally solved the problem by first explicitly identifying the code blocks and then by composing them into a solution.

Finally, Bard performed better on some problems than others. The multiple regression analysis which modeled both the problem and the representation as independent variables revealed a significant overall effect (F-statistic = 10.02, $p < 0.001$) with an adjusted R^2 of 0.3776, indicating that the predictors collectively explain a substantial portion of the variance in scores. After correcting for multiple comparisons, the coefficients for individual predictors suggested that EricsonFig4 (Estimate = -0.35, $p < 0.01$), HaynesFig2 (Estimate = 0.40, $p < 0.001$), and ProblemHou2022 (Estimate = 0.40, $p < 0.001$) had significant effects on scores compared with the baseline of Ericson2022figure2. While various factors may contribute to these performance differences, the findings indicate that performance indeed varies across different problem types.

4.2 RQ 2: Effects of Visual Representation

4.2.1 Effects of Visual Representation on GPT-4V Performance. GPT-4V appeared to recognize the text correctly without refusals (e.g. "I can not seem to extract the text from the image.") The only issues we observed were cases where GPT-4V hallucinated code that was nearly semantically identical as shown in Figure 3. This suggests the code was recognized and understood but there may have been issues in terms of how it was encoded in the vector space or how the generated text was produced.

4.2.2 Effects of Visual Representation on Bard Performance. Bard struggled with image-to-text extraction. In 7.5% of all trials, Bard refused to solve the problem. For the Ericson2022figure4 problem in the Epplets format, Bard refused to attempt to solve the problem in all 5 trials. For those cases, the response included a variation of the phrase "I'm only a language model," which could mislead users to believe it does not have capability to process image data. Based on the multiple regression analysis described earlier, which accounted for multiple comparisons, visual representation had an impact on the success rate, with the JS Parsons representation showing significantly higher scores than Epplets (Estimate=0.33, $p < 0.01$). In the worst case, Bard was only able to solve 50% of the problems when represented in the Epplets visual format. In cases such as in Figure 4, Bard failed to solve the problem because it hallucinates incorrect code blocks, rewriting lines such as "if day == 0 or day == 6" into "if day > 5." Given these are both conditional statements, this suggests a semantic understanding issue rather than a problem with text recognition.

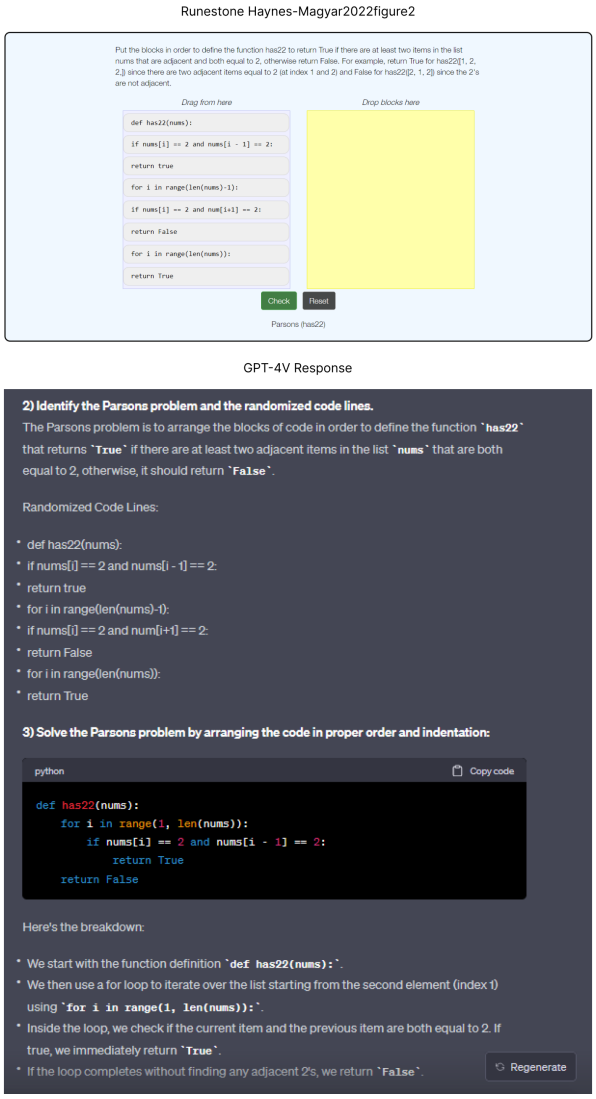

Figure 3: Runestone visual representation of the Haynes-Magyar2022figure2 Parsons problem. In its response, GPT-4V incorrectly alters a code block to "for i in range(1, len(nums))."

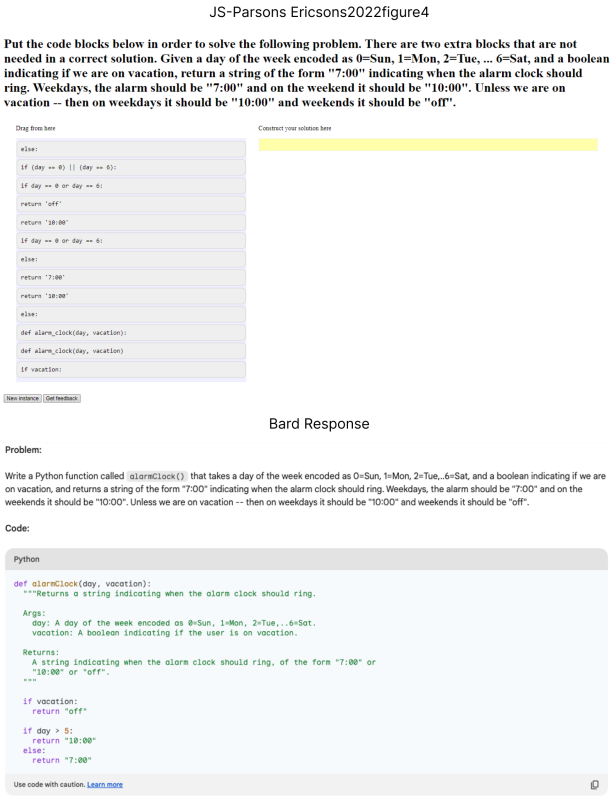

Figure 4: JS-Parsons visual representation of the Ericson2022figure4 Parsons problem. In its response, Bard extracted the text incorrectly and generated its own code blocks.

5 DISCUSSION

5.1 RQ1: Correctness of Generated Solutions

The results of our study suggest that large multimodal models (LMMs) are already capable of solving visual programming problems with remarkable proficiency. We investigated this capability with Parsons problems presented in four visually diverse representations. There are two main aspects of solving visual programming problems: (1) analyzing the image and (2) solving the coding problem. GPT-4V performed well across the range of visual presentation formats, despite variations in fonts, font sizes, backgrounds, colors, interaction styles, and text location.

GPT-4V was able to solve 96.7% of all Parsons problems on its second attempt, providing consistently perfect indentation and syntax. Bard was only able to solve 69.2% problems, which was still substantially higher than the 51% correctness achieved in previous studies when OpenAI Codex was used [54]. This is noteworthy, because unlike in that prior work, all of the Parsons problems that we evaluated were presented to the model without the help of pre-indentation. In addition to using a more recent model, our prompt was also more explicit, with included context, task, and step-by-step instructions. During prompt engineering, the performance was closer to 50% if we simply asked GPT-4V to "solve the Parsons problem in the provided image." Another observation is that the type of problem, including problems that contained distractors, did not appear to negatively impact the performance of GPT-4V. Bard encountered difficulties with specific problems.

5.2 RQ2: Effects of Visual Representation

The Parsons problems were presented in multiple diverse visual representations. While this did not appear to affect GPT-4V, we observed some visual representations were easier for Bard than others. For Bard, problems presented in Epplets format had the lowest performance. While it is unclear what about Epplets led to the lower performance, visual inspection suggests that it has the smallest font size and more visual stimuli than the others. The interface also contained more text input windows than the other

representations. There was a window containing the whole program, the fragment bank, the solution area, and two other irrelevant windows at the bottom containing additional text. To add to the complexity, each window had both horizontal and vertical scroll bars. There were buttons at the bottom right of the image indicate actions for quitting, submitting, and navigating to a different problem. A menu bar at the top left included various options, editing, viewing, formatting, and help drop-downs. In addition to the input controls, instructions related to the program itself were visible in the interface. There were two timers in the bottom right of the input that indicate the elapsed time and remaining time. Finally, a message related to grading appears in the bottom right window. GPT-4V did not appear to be affected by the visual representation, but these visual elements negatively affect the performance of Bard.

5.3 Implications for Assessment

Recently, researchers and practitioners have been investigating techniques to address unsanctioned use of large language models in computing classrooms, such as presenting problems visually [12]. However, our results suggest that LMMs are now capable of solving programming problems that are represented visually. This introduces a new threat toward traditional assessment and challenges educators to reconsider assessment in the age of generative AI.

There appear to be two paths forward. The first is adopting adversarial methods toward assessment which could include "data poisoning" attacks [63]. Data poisoning or 'AI poisoning' is a technique to "manipulate training data to introduce unexpected behaviors into machine learning models at training time." These attacks focus on training data, but they could also focus on input. Our results show some preliminary evidence for this approach. Since Bard faced challenges with Epplets due to visual complexity, creating visually intricate problems with distracting elements might deceive models to perform poorly. However, this may also confuse students and negatively impact learning. A variation could be to inject noise that is not perceptible to students, but that would cause the model to perform poorly. This is likely a temporary solution given the ongoing improvements in model performance.

A second path is to consider alternative forms of assessment and relying on intrinsic motivation to ensure students do not use models in unsanctioned ways. This is an exciting approach because ungrading [4, 8, 65], culturally responsive computing [62], and other alternative pedagogical styles [40, 47] have shown great promise, but have not yet been widely adopted. Perhaps these new threats to assessment may challenge educators to try them in their classrooms.

5.4 Implications for Pedagogy

This paper's results add to a growing list of examples showcasing how LLMs and LMMs can effectively solve problems for students [22, 35, 37, 42, 48, 49, 53, 58, 59, 61, 70]. While there are clear implications for assessment, there are also considerations for pedagogy, especially around the ethical use of generative AI.

Our results show that some models perform better than others. Based on our prompt engineering, we also reiterate previous work about the importance of identifying effective prompts. Taken together, these observations suggest that some students may be able to use these models to great effect, while others may struggle to use them to get the correct answer. This suggests that there is a potential for unequal benefits. Additional work is needed to understand student prompting behaviors [73].

6 LIMITATIONS

There are a few limitations that must be considered when evaluating the results presented in this paper. First is the number of visual representations presented and second is the selection of Parsons problems rather than other visual programming problems. Additionally, while we randomized code blocks for each problem to avoid ordering effects, unlike Reeves et al., we did not randomize code blocks each time the model was prompted. It is unclear if there was an effect on our results.

The first limitation was related to the availability and accessibility of diverse visual representations of Parsons problems. This constraint stemmed from several factors, including the scarcity of open-source Parson problem generators and the lack of existing visual Parson problem datasets. Another limitation is our approach to obtaining visual representations, which relied on capturing screenshots and downloading Parson problems. This led to some minor variations in PNG aspect ratios. Despite efforts to standardize, such as downloading images in the same file formats and with the same pixel density, these minor discrepancies remained. While this reflects a good sampling of the real-world diversity of visual representations students might encounter, it may introduce the possibility that GPT-4V's performance varies depending on additional factors such as aspect ratio or pixel density.

The second limitation is our focus solely on Parson problems as a form of visual programming problems. While these problems have provided valuable insights, it is essential to recognize that there are many other visual programming problems, such as UML diagrams, input-output tables, and data-structures problems involving graphs and trees. As future work, we are evaluating these alternative problems to provide a more comprehensive understanding of the performance of these models on diverse visual programming problems. Furthermore, we acknowledge that our study exclusively relied on only two LMMs, GPT-4V and Bard. While these models are extremely popular, it is essential to recognize that other LMMs are available [25, 43, 69]. Assessing the performance of LMMs would have offered a broader perspective on their respective strengths and weaknesses. Such an approach could have illuminated how these models perform in various scenarios and their overall efficacy in the broader educational landscape.

7 CONCLUSION

In conclusion, the rise of LLMs has prompted educators to reassess their assessment methods and devise new ways of ensuring students 'do the work.' To address this issue, researchers and practitioners have proposed assigning visual programming problems to make it harder for students to use LLMs. Our study examined the performance of the recently released large multimodal models (LMMs) on multiple Parsons problems presented across diverse visual representations. GPT-4V and Bard were able to solve 96.7% and 69.2% respectively. The remarkable proficiency demonstrated by GPT-4V in tackling these problems across differences in visual presentation, challenges the idea that using visual programming problems will

be a panacea for maintaining traditional assessment in the era of Generative AI. As we navigate this evolving landscape, it becomes clear that broader pedagogical considerations are essential to effectively address the impact of these models and ensure a holistic and adaptive approach to teaching and learning. First, we recommend that instructors reconsider their assessment practices in light of the growing number of AI capabilities, and second, we recommend instructors consider the wide range of performance obtained by LMMs based on how they are prompted and which model is selected. These provide two interesting directions for future work.

REFERENCES

[1] A.T.M. Bari, Alessio Gaspar, R. Wiegand, Jennifer Albert, Anthony Bucci, and Amruth Kumar. 2019. EvoParsons: design, implementation and preliminary evaluation of evolutionary Parsons puzzle. *Genetic Programming and Evolvable Machines* 20 (06 2019). https://doi.org/10.1007/s10710-019-09343-7

[2] Shraddha Barke, Michael B James, and Nadia Polikarpova. 2022. Grounded Copilot: How Programmers Interact with Code-Generating Models. *arXiv preprint arXiv:2206.15000* (2022).

[3] Brett A Becker, Paul Denny, James Finnie-Ansley, Andrew Luxton-Reilly, James Prather, and Eddie Antonio Santos. 2023. Programming is hard-or at least it used to be: Educational opportunities and challenges of ai code generation. In *Proceedings of the 54th ACM Technical Symposium on Computer Science Education V. 1*. 500–506.

[4] Susan Debra Blum. 2020. *Ungrading: Why rating students undermines learning (and what to do instead)*. West Virginia University Press.

[5] Peter Brusilovsky, Stephen Edwards, Amruth Kumar, Lauri Malmi, Luciana Benotti, Duane Buck, Petri Ihantola, Rikki Prince, Teemu Sirkiä, Sergey Sosnovsky, Jaime Urquiza-Fuentes, Arto Hellas, and Michael Wollowski. 2014. Increasing Adoption of Smart Learning Content for Computer Science Education. *ITiCSE-WGR 2014 - Working Group Reports of the 2014 Innovation and Technology in Computer Science Education Conference*. https://doi.org/10.1145/2713609.2713611

[6] Michael B. Cahapay. 2021. Problems Encountered by College Students in Online Assessment Amid COVID-19 Crisis: A Case Study. *SSRN Electronic Journal* (2021). https://doi.org/10.2139/ssrn.3791771

[7] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. 2021. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374* (2021).

[8] Gary Chu. 2020. The Point-less Classroom: A Math Teacher's Ironic Choice in not Calculating Grades. In *Ungrading: Why rating students undermines learning (and what to do instead)*, Susan Debra Blum (Ed.). West Virginia University Press, Chapter 10, 161–170.

[9] Simon Coghlan, Tim Miller, and Jeannie Paterson. 2021. Good Proctor or "Big Brother"? Ethics of Online Exam Supervision Technologies. *Philosophy & Technology* 34, 4 (Aug. 2021), 1581–1606. https://doi.org/10.1007/s13347-021-00476-1

[10] Dwight Davis, J Kevin Dorsey, Ronald D Franks, Paul R Sackett, Cynthia A Searcy, and Xiaohui Zhao. 2013. Do racial and ethnic group differences in performance on the MCAT exam reflect test bias? *Academic Medicine* 88, 5 (2013), 593–602.

[11] Paul Denny, Viraj Kumar, and Nasser Giacaman. 2023. Conversing with Copilot: Exploring Prompt Engineering for Solving CS1 Problems Using Natural Language. In *Proceedings of the 54th ACM Technical Symposium on Computer Science Education V. 1* (Toronto ON, Canada) *(SIGCSE 2023)*. Association for Computing Machinery, New York, NY, USA, 1136–1142. https://doi.org/10.1145/3545945.3569823

[12] Paul Denny, Juho Leinonen, James Prather, Andrew Luxton-Reilly, Thezyrie Amarouche, Brett A. Becker, and Brent N. Reeves. 2023. Promptly: Using Prompt Problems to Teach Learners How to Effectively Utilize AI Code Generators. arXiv:2307.16364 [cs.HC]

[13] Paul Denny, Andrew Luxton-Reilly, and Beth Simon. 2008. Evaluating a new exam question: Parsons problems. In *Proceedings of the fourth international workshop on computing education research*. 113–124.

[14] Yuemeng Du, Andrew Luxton-Reilly, and Paul Denny. 2020. A review of research on Parsons problems. In *Proc. of the Twenty-Second Australasian Computing Education Conf.* 195–202.

[15] Barbara Ericson, Mark Guzdial, and Briana Morrison. 2015. Analysis of Interactive Features Designed to Enhance Learning in an Ebook. 169–178. https://doi.org/10.1145/2787622.2787731

[16] Barbara J Ericson, Paul Denny, James Prather, Rodrigo Duran, Arto Hellas, Juho Leinonen, Craig S Miller, Briana B Morrison, Janice L Pearce, and Susan H Rodger. 2022. Parsons problems and beyond: Systematic literature review and empirical study designs. *Proceedings of the 2022 Working Group Reports on Innovation and Technology in Computer Science Education* (2022), 191–234.

[17] Barbara J. Ericson, Mark J. Guzdial, and Briana B. Morrison. 2015. Analysis of Interactive Features Designed to Enhance Learning in an Ebook. In *Proceedings of the Eleventh Annual International Conference on International Computing Education Research* (Omaha, Nebraska, USA) *(ICER '15)*. Association for Computing Machinery, New York, NY, USA, 169–178. https://doi.org/10.1145/2787622.2787731

[18] Barbara J. Ericson, Lauren E. Margulieux, and Jochen Rick. 2017. Solving Parsons Problems versus Fixing and Writing Code. In *Proceedings of the 17th Koli Calling International Conference on Computing Education Research* (Koli, Finland) *(Koli Calling '17)*. Association for Computing Machinery, 20–29. https://doi.org/10.1145/3141880.3141895

[19] Barbara J. Ericson and Bradley N. Miller. 2020. Runestone: A Platform for Free, On-Line, and Interactive Ebooks. In *Proceedings of the 51st ACM Technical Symposium on Computer Science Education* (Portland, OR, USA) *(SIGCSE '20)*. Association for Computing Machinery, New York, NY, USA, 1012–1018. https://doi.org/10.1145/3328778.3366950

[20] Barbara J Ericson, Janice L Pearce, Susan H Rodger, Andrew Csizmadia, Rita Garcia, Francisco J Gutierrez, Konstantinos Liaskos, Aadarsh Padiyath, Michael James Scott, David H Smith, et al. 2023. Conducting multi-institutional studies of Parsons problems. In *Proceedings of the 2023 Conference on Innovation and Technology in Computer Science Education V. 2*. 571–572.

[21] Barbara J. Ericson, Kantwon Rogers, Miranda Parker, Briana Morrison, and Mark Guzdial. 2016. Identifying Design Principles for CS Teacher Ebooks through Design-Based Research. In *Proceedings of the 2016 ACM Conference on International Computing Education Research* (Melbourne, VIC, Australia) *(ICER '16)*. Association for Computing Machinery, New York, NY, USA, 191–200. https://doi.org/10.1145/2960310.2960335

[22] Zhiyu Fan, Xiang Gao, Martin Mirchev, Abhik Roychoudhury, and Shin Hwei Tan. 2023. Automated repair of programs from large language models. In *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*. IEEE, 1469–1481.

[23] James Finnie-Ansley, Paul Denny, Brett A. Becker, Andrew Luxton-Reilly, and James Prather. 2022. The Robots Are Coming: Exploring the Implications of OpenAI Codex on Introductory Programming. In *Australasian Computing Education Conf.* (Virtual Event, Australia) *(ACE '22)*. ACM, New York, NY, USA, 10–19. https://doi.org/10.1145/3511861.3511863

[24] Alessio Gaspar, Dmytro Vitel, and A.T.M. Bari. 2019. Lessons Learned from Available Parsons Puzzles Software. https://doi.org/10.18260/1-2--33060

[25] Tao Gong, Chengqi Lyu, Shilong Zhang, Yudong Wang, Miao Zheng, Qian Zhao, Kuikun Liu, Wenwei Zhang, Ping Luo, and Kai Chen. 2023. Multimodal-gpt: A vision and language model for dialogue with humans. *arXiv preprint arXiv:2305.04790* (2023).

[26] Carina S González-González, Alfonso Infante-Moro, and Juan C Infante-Moro. 2020. Implementation of e-proctoring in online teaching: A study about motivational factors. *Sustainability* 12, 8 (2020), 3488.

[27] Kyle James Harms, Jason Chen, and Caitlin L. Kelleher. 2016. Distractors in Parsons Problems Decrease Learning Efficiency for Young Novice Programmers. In *Proceedings of the 2016 ACM Conference on International Computing Education Research* (Melbourne, VIC, Australia) *(ICER '16)*. Association for Computing Machinery, 241–250. https://doi.org/10.1145/2960310.2960314

[28] Juha Helminen, Petri Ihantola, Ville Karavirta, and Lauri Malmi. 2012. How Do Students Solve Parsons Programming Problems? An Analysis of Interaction Traces. In *Proceedings of the Ninth Annual International Conference on International Computing Education Research* (Auckland, New Zealand) *(ICER '12)*. Association for Computing Machinery, New York, NY, USA, 119–126. https://doi.org/10.1145/2361276.2361300

[29] Jeffrey D Holmes. 2021. The Bad Test-Taker Identity. *Teaching of psychology* 48, 4 (2021), 293–299.

[30] Irene Hou, Sophia Mettille, Owen Man, Zhuo Li, Cynthia Zastudil, and Stephen MacNeil. 2024. The Effects of Generative AI on Introductory Students' Help-Seeking Preferences. In *Australasian Computing Education Conference (ACE '24)*. ACM.

[31] Petri Ihantola, Juha Helminen, and Ville Karavirta. 2013. How to study programming on mobile touch devices. In *Proceedings of the 13th Koli Calling International Conference on Computing Education Research*. ACM. https://doi.org/10.1145/2526968.2526974

[32] Petri Ihantola, Juha Helminen, and Ville Karavirta. 2013. How to Study Programming on Mobile Touch Devices: Interactive Python Code Exercises. In *Proceedings of the 13th Koli Calling International Conference on Computing Education Research* (Koli, Finland) *(Koli Calling '13)*. Association for Computing Machinery, New York, NY, USA, 51–58. https://doi.org/10.1145/2526968.2526974

[33] Petri Ihantola and Ville Karavirta. 2011. Two-Dimensional Parson's Puzzles: The Concept, Tools, and First Observations. *Journal of Information Technology Education: Innovations in Practice* 10 (01 2011), 1–14. https://doi.org/10.28945/1394

[34] Petri Ihantola and Ville Karavirta. 2011. Two-Dimensional Parson's Puzzles: The Concept, Tools, and First Observations. *Journal of Information Technology Education: Innovations in Practice* 10 (2011), 119–132.

[35] Nan Jiang, Kevin Liu, Thibaud Lutellier, and Lin Tan. 2023. Impact of code language models on automated program repair. *arXiv:2302.05020* (2023).

[36] Ishika Joshi, Ritvik Budhiraja, Harshal Dev, Jahnvi Kadia, Muhammad Osama, Ataullah, Sayan Mitra, and Dhruv Kumar. 2023. ChatGPT and the Future of Undergraduate Computer Science: Challenges, Opportunities and Recommendations. https://api.semanticscholar.org/CorpusID:258741417

[37] Charles Koutcheme, Sami Sarsa, Juho Leinonen, Arto Hellas, and Paul Denny. 2023. Automated Program Repair Using Generative Models for Code Infilling. In *International Conference on Artificial Intelligence in Education*. Springer, 798–803.

[38] Amruth Kumar. 2017. The Effect of Providing Motivational Support in Parsons Puzzle Tutors. 528–531. https://doi.org/10.1007/978-3-319-61425-0_56

[39] Amruth N. Kumar. 2018. Epplets: A Tool for Solving Parsons Puzzles. In *Proceedings of the 49th ACM Technical Symposium on Computer Science Education* (Baltimore, Maryland, USA) *(SIGCSE '18)*. Association for Computing Machinery, New York, NY, USA, 527–532. https://doi.org/10.1145/3159450.3159576

[40] Celine Latulipe, N Bruce Long, and Carlos E Seminario. 2015. Structuring flipped classes with lightweight teams and gamification. In *Proceedings of the 46th ACM Technical Symposium on Computer Science Education*. 392–397.

[41] Sam Lau and Philip J. Guo. 2023. From 'Ban It Till We Understand It' to "Resistance is Futile": How University Programming Instructors Plan to Adapt as More Students Use AI Code Generation and Explanation Tools such as ChatGPT and GitHub Copilot. In *Proceedings of the 2023 ACM Conference on International Computing Education Research V.1 (ICER '23 V1)*. ACM. https://doi.org/10.1145/3568813.3600138

[42] Juho Leinonen, Paul Denny, Stephen MacNeil, Sami Sarsa, Seth Bernstein, Joanne Kim, Andrew Tran, and Arto Hellas. 2023. Comparing Code Explanations Created by Students and Large Language Models. *arXiv preprint arXiv:2304.03938* (2023).

[43] Haotian Liu, Chunyuan Li, Qingyang Wu, and Yong Jae Lee. 2023. Visual instruction tuning. *arXiv preprint arXiv:2304.08485* (2023).

[44] Yuliang Liu, Zhang Li, Hongliang Li, Wenwen Yu, Mingxin Huang, Dezhi Peng, Mingyu Liu, Mingrui Chen, Chunyuan Li, Lianwen Jin, et al. 2023. On the hidden mystery of ocr in large multimodal models. *arXiv preprint arXiv:2305.07895* (2023).

[45] Stephen MacNeil, Paul Denny, Andrew Tran, Juho Leinonen, Seth Bernstein, Arto Hellas, Sami Sarsa, and Joanne Kim. 2023. Decoding Logic Errors: A Comparative Study on Bug Detection by Students and Large Language Models. *arXiv preprint arXiv:2311.16017* (2023).

[46] Stephen MacNeil, Joanne Kim, Juho Leinonen, Paul Denny, Seth Bernstein, Brett A. Becker, Michel Wermelinger, Arto Hellas, Andrew Tran, Sami Sarsa, James Prather, and Viraj Kumar. 2023. The Implications of Large Language Models for CS Teachers and Students. In *Proceedings of the 54th ACM Technical Symposium on Computer Science Education V. 2 (SIGCSE 2023)*. Association for Computing Machinery, 1255. https://doi.org/10.1145/3545947.3573358

[47] Stephen MacNeil, Celine Latulipe, Bruce Long, and Aman Yadav. 2016. Exploring lightweight teams in a distributed learning environment. In *Proceedings of the 47th ACM Technical Symposium on Computing Science Education*. 193–198.

[48] Stephen MacNeil, Andrew Tran, Arto Hellas, Joanne Kim, Sami Sarsa, Paul Denny, Seth Bernstein, and Juho Leinonen. 2023. Experiences from Using Code Explanations Generated by Large Language Models in a Web Software Development E-Book. In *Proc. SIGCSE'23*. ACM, 6 pages.

[49] Stephen MacNeil, Andrew Tran, Dan Mogil, Seth Bernstein, Erin Ross, and Ziheng Huang. 2022. Generating Diverse Code Explanations Using the GPT-3 Large Language Model. In *Proc. of the 2022 ACM Conf. on Int. Computing Education Research - Volume 2*. ACM, 37–39.

[50] Dale Parsons and Patricia Haden. 2006. Parson's programming puzzles: a fun and effective learning tool for first programming courses. In *Proceedings of the 8th Australasian Conference on Computing Education-Volume 52*. 157–163.

[51] James Prather, Paul Denny, Juho Leinonen, Brett A Becker, Ibrahim Albluwi, Michael E Caspersen, Michelle Craig, Hieke Keuning, Natalie Kiesler, Tobias Kohn, et al. 2023. Transformed by Transformers: Navigating the AI Coding Revolution for Computing Education: An ITiCSE Working Group Conducted by Humans. In *Proceedings of the 2023 Conference on Innovation and Technology in Computer Science Education V. 2*. 561–562.

[52] James Prather, Paul Denny, Juho Leinonen, Brett A Becker, Ibrahim Albluwi, Michelle Craig, Hieke Keuning, Natalie Kiesler, Tobias Kohn, Andrew Luxton-Reilly, et al. 2023. The Robots are Here: Navigating the Generative AI Revolution in Computing Education. *arXiv preprint arXiv:2310.00658* (2023).

[53] Ben Puryear and Gina Sprint. 2022. Github copilot in the classroom: learning to code with AI assistance. *Journal of Computing Sciences in Colleges* 38, 1 (2022), 37–47.

[54] Brent Reeves, Sami Sarsa, James Prather, Paul Denny, Brett A Becker, Arto Hellas, Bailey Kimmel, Garrett Powell, and Juho Leinonen. 2023. Evaluating the performance of code generation models for solving Parsons problems with small prompt variations. In *Proceedings of the 2023 Conference on Innovation and Technology in Computer Science Education V. 1*. 299–305.

[55] Jürgen Rudolph, Samson Tan, and Shannon Tan. 2023. ChatGPT: Bullshit spewer or the end of traditional assessments in higher education? *Journal of Applied Learning and Teaching* 6, 1 (2023).

[56] Vinu Sankar Sadasivan, Aounon Kumar, Sriram Balasubramanian, Wenxiao Wang, and Soheil Feizi. 2023. Can ai-generated text be reliably detected? *arXiv preprint arXiv:2303.11156* (2023).

[57] Sami Sarsa, Paul Denny, Arto Hellas, and Juho Leinonen. 2022. Automatic Generation of Programming Exercises and Code Explanations Using Large Language Models. In *Proc. of the 2022 ACM Conf. on Int. Computing Education Research - Volume 1*. ACM, 27–43.

[58] Jaromir Savelka, Arav Agarwal, Marshall An, Chris Bogart, and Majd Sakr. 2023. Thrilled by Your Progress! Large Language Models (GPT-4) No Longer Struggle to Pass Assessments in Higher Education Programming Courses. *The 19th ACM Conference on International Computing Education Research (ICER)* (2023).

[59] Jaromir Savelka, Arav Agarwal, Christopher Bogart, and Majd Sakr. 2023. Large language models (gpt) struggle to answer multiple-choice questions about code. *arXiv preprint arXiv:2303.08033* (2023).

[60] Jaromir Savelka, Arav Agarwal, Christopher Bogart, Yifan Song, and Majd Sakr. 2023. Can Generative Pre-trained Transformers (GPT) Pass Assessments in Higher Education Programming Courses? arXiv:2303.09325

[61] Jaromir Savelka, Arav Agarwal, Christopher Bogart, Yifan Song, and Majd Sakr. 2023. Can Generative Pre-trained Transformers (GPT) Pass Assessments in Higher Education Programming Courses? *arXiv preprint arXiv:2303.09325* (2023).

[62] Kimberly A Scott, Kimberly M Sheridan, and Kevin Clark. 2015. Culturally responsive computing: A theory revisited. *Learning, Media and Technology* 40, 4 (2015), 412–436.

[63] Shawn Shan, Wenxin Ding, Josephine Passananti, Haitao Zheng, and Ben Y Zhao. 2023. Prompt-Specific Poisoning Attacks on Text-to-Image Generative Models. *arXiv preprint arXiv:2310.13828* (2023).

[64] Teemu Sirkiä. 2016. Combining Parson's Problems with Program Visualization in CS1 Context. In *Proceedings of the 16th Koli Calling International Conference on Computing Education Research* (Koli, Finland) *(Koli Calling '16)*. Association for Computing Machinery, New York, NY, USA, 155–159. https://doi.org/10.1145/2999541.2999558

[65] Scott Spurlock. 2023. Improving Student Motivation by Ungrading. In *Proceedings of the 54th ACM Technical Symposium on Computer Science Education V. 1*.

[66] Anselm L Strauss and Juliet Corbin. 2004. Open coding. *Social research methods: A reader* (2004), 303–306.

[67] Teo Susnjak. 2022. ChatGPT: The End of Online Exam Integrity? arXiv:2212.09292

[68] Andrew Tran, Kenneth Angelikas, Egi Rama, Chiku Okechukwu, David H Smith IV, and Stephen MacNeil. 2023. Generating Multiple Choice Questions for Computing Courses using Large Language Models. (2023).

[69] Jiaqi Wang, Zhengliang Liu, Lin Zhao, Zihao Wu, Chong Ma, Sigang Yu, Haixing Dai, Qiushi Yang, Yiheng Liu, Songyao Zhang, et al. 2023. Review of large vision models and visual prompt engineering. *arXiv preprint arXiv:2307.00855* (2023).

[70] Michel Wermelinger. 2023. Using GitHub Copilot to Solve Simple Programming Problems. (2023).

[71] Benjamin Xie, Matt J Davidson, Baker Franke, Emily McLeod, Min Li, and Amy J Ko. 2021. Domain Experts' Interpretations of Assessment Bias in a Scaled, Online Computer Science Curriculum. In *Proceedings of the Eighth ACM Conference on Learning@ Scale*. 77–89.

[72] Iman YeckehZaare, Paul Resnick, and Barbara Ericson. 2019. A Spaced, Interleaved Retrieval Practice Tool That is Motivating and Effective. In *Proceedings of the 2019 ACM Conference on International Computing Education Research* (Toronto ON, Canada) *(ICER '19)*. Association for Computing Machinery, New York, NY, USA, 71–79. https://doi.org/10.1145/3291279.3339411

[73] JD Zamfirescu-Pereira, Richmond Y Wong, Bjoern Hartmann, and Qian Yang. 2023. Why Johnny can't prompt: how non-AI experts try (and fail) to design LLM prompts. In *Proceedings of the 2023 CHI Conference on Human Factors in Computing Systems*. 1–21.

[74] Cynthia Zastudil, Magdalena Rogalska, Christine Kapp, Jennifer Vaughn, and Stephen MacNeil. 2023. Generative AI in Computing Education: Perspectives of Students and Instructors. *arXiv preprint arXiv:2308.04309* (2023).